FloodSense

Warn people hours before the flood. Build plan for Environmental Hacks (WeMakeDevs x AWS, Bharat Builds Event 02)

Track: Heat and Water. Team of 2. Event window: Thu 8 Oct to Sun 11 Oct 2026. Judging is online only, from the submission. Prepared 9 Oct 2026.

1. How we are scored (30 points)

Criterion	Points	Typical entry	Our lever
AWS Usage	10	5	Biggest gap. One third of the total. Agent plus many services, each visibly doing a job in the video.
Idea and Impact	8	6	Answer the second half: what changes for the people living with it.
Execution	4	3	One end-to-end flow that works on camera: photo in, map changes, SMS arrives.
Demo Video	4	4	3 minutes. Judges score only what the video shows.
Design and Usability	4	2	A stranger must know what to do. This is where most teams lose points.

What the organisers and past winners tell us

- The announcement blog says most projects lose points on 'what changes for the people living with it'. Our pitch is a decision tool, not a map.
- Rules: judges score only the submission. No live demo. A feature that is not in the video does not exist. AWS must be shown in the video, not just named in the writeup.
- Be upfront about where data comes from and how fresh it is. The organisers say this explicitly.
- The First Commit Build It winner (Beacon Night Shift) used seven AWS services with one named job each: Bedrock, Step Functions, Transcribe, Polly, Lambda, DynamoDB, CloudWatch. Agents dominated the leaderboard. Two of three winners were hosted on Amplify.
- The organisers literally listed 'a monsoon commute helper that identifies which routes flood first, using resident reports rather than sensors' as a sample idea. The lane is wanted, so others will build a flood map. We win on the parts they will not have.
- Repo history must match event dates. Nothing committed before 8 Oct. AI coding tools are allowed but must be listed in the writeup.
- Submission: public repo, YouTube video (public or unlisted, 3 min max), short writeup (problem, build, where AWS fits). Optional blog on AWS Builder Center: top 5 blogs win AirPods.

2. The idea in one paragraph

In heavy monsoon rain a street can fill in an hour, and people find out after it has happened. Rain alone does not predict it: 50 mm is fine on a high street with a clear drain and floods a low street with a blocked one. FloodSense couples three things per road segment: the rain forecast, how low-lying the segment is (elevation), and whether nearby drains are blocked right now (citizen photos verified by AI). It then tells two people exactly what to do: the commuter gets 'Station Road likely floods by 9:10, leave by 8:15 or use Gandhi Marg', and the drain engineer gets 'clean drain D-14 first, it turns Station Road from red to yellow for tomorrow's rain'.

Core rule (the coupling)

Heavy rain + low-lying segment + blocked drain nearby = RED. Same rain, high segment, clear drain = GREEN. Every colour comes with a one-sentence reason.

3. Final scope

Area

Central Delhi, a few square km containing the well-known hotspots: Minto Bridge, ITO, Connaught Place area. These named hotspots are our validation set. A full official list is not public, so we validate against the roughly ten named in the press and say so on screen.

Two users, three screens

- **Commuter screen (Anita).** Map with every road segment green, amber or red for the next 0 to 6 hours. Tap a segment to see the reason. One alert card with a time and an alternative road. Hindi and English toggle. Optional Polly audio of the alert.
- **Report screen (citizen).** Take a photo of a blocked drain or waterlogging. It auto-geotags, the agent verifies it and rates severity, and the screen shows the consequence immediately: 'this report raised Station Road from amber to red'.
- **Engineer screen (Mr. Das).** Ranked drain-cleaning list. Each row has a counterfactual: 'Clean D-14 first. Station Road goes red to amber for tomorrow's 60 mm.' One tap marks it cleaned and the map updates. This single line proves the coupling and answers 'what changes for people'.
- **Storm replay toggle.** Replay a real 2026 Delhi storm from Open-Meteo's historical archive, clearly labelled as a replay on screen, so the video has drama even if Sunday is dry.
- **Honesty footer.** Data sources and freshness shown in the app: rain from Open-Meteo (hourly), elevation from Copernicus DEM, drains from OpenStreetMap plus citizen reports.

Cut (do not build)

- Self-learning rule updates. Replaced by: post-storm photos are logged as confirmed flooding and the app shows a hit-rate number.
- Ambulance and traffic-police views.
- Real routing. Alternative road is a precomputed field per segment.
- Live sensors. We say clearly we use forecasts and resident reports, which is what the organisers suggested.
- Anything not in the video.

4. AWS architecture (every service has a job the video can show)

#	Service / tool	Job in FloodSense	Shown in video as
1	Strands Agents SDK (AWS open source)	The FloodSense agent with four tools: fetch forecast, score segments, rank drains, draft alert. Satisfies the mandatory Build It requirement on its own.	Agent trace while a photo is processed
2	Amazon Bedrock (Nova Lite / Nova Pro vision)	Verifies photos (blocked drain / waterlogging / unrelated), rates severity, writes the plain-language reason for each colour.	Photo result card
3	Bedrock AgentCore Runtime	Hosts the agent. Time-box to 2 hours on day one. Fallback: run the agent inside Lambda.	Console flash, 3 s
4	Lambda + API Gateway	Backend API for the app.	Architecture diagram
5	EventBridge Scheduler	Rescores every segment every hour against the fresh forecast.	Console flash, 3 s
6	Step Functions (stretch)	Alert workflow: score, threshold check, send, log. Same pattern the Build It winner used.	State machine graph
7	DynamoDB	Segments, drains, reports, alerts.	Architecture diagram
8	S3	Photo uploads via presigned URLs, cached elevation tiles.	Architecture diagram
9	Registry of Open Data on AWS: Copernicus DEM	Elevation per segment (sink depth). AWS as a data source, which the organisers recommended.	Data sources slide
10	Amazon SNS	SMS alert to the commuter. Sandbox with verified numbers, no DLT needed for the demo.	SMS arriving on a phone on camera
11	Amazon Location Service	Map tiles via MapLibre, so even the map is AWS.	The map itself
12	Amplify Hosting	Deployed Next.js frontend. Gives the amplifyapp.com live link judges expect.	Live URL in writeup and video
13	CloudWatch	Dashboard: reports processed, alerts sent, rescoring runs.	Dashboard, 5 s
14	Amazon Polly	Reads the Hindi alert aloud for low-literacy users. One API call.	Audio plays in app

Non-AWS pieces, declared openly: Open-Meteo (free, no key, hourly forecast and historical archive) for rain, OpenStreetMap roads and storm-drain nodes pulled once via Overpass and cached as JSON.

5. Design and usability rules (4 points, the usual weak spot)

- Mobile-first. Three screens. One primary action per screen, nothing else on it.
- Tailwind plus shadcn components. One accent colour. Green, amber and red are reserved for risk only.
- Every red segment has a reason sentence. Every alert has a time and an alternative.
- Empty states and loading states exist. The judge will open it on a laptop with zero reports in it.

- Hindi and English toggle in the header.
- Saturday night: hand the deployed link to a friend and watch them use it without help. Fix whatever confuses them.

6. Who does what

Backend + AWS (Devansh)	Frontend + video (teammate)
AWS account, Bedrock model access, SNS sandbox numbers, IAM, billing alert	Next.js app skeleton on Amplify Hosting
DynamoDB tables, Lambda + API Gateway	Amazon Location Service map with MapLibre, segment colouring
Strands agent and its four tools	Commuter, Report and Engineer screens with fake data first
Copernicus DEM tile pull, per-segment sink depth	Photo capture and upload to S3 via presigned URL
Open-Meteo wiring, scoring engine, counterfactual	Hindi/English toggle, empty and loading states
EventBridge, CloudWatch, SNS send, Polly	Storm replay UI, architecture diagram, video, blog

Schedule

When	Goal
Fri 9 Oct (today) + Sat morning	Backend: account and access set up, tables, API, agent with scoring tool, DEM and forecast wired. Frontend: three screens working on fake data, map rendering, deployed once on Amplify.
Sat 10 Oct	Connect everything. Photo flow end to end. SMS on a real phone. Engineer counterfactual. Storm replay. EventBridge and CloudWatch. Step Functions only if ahead of time. Friend test at night.
Sun 11 Oct morning	Video and blog only. Nothing new gets built. Check the deadline time on the Schedule page the night before.

7. Day-one setup checklist

AWS (blocks everything, do first)

1. Create the AWS account at aws.amazon.com/free with a card. Pick the free plan. Allow 30 to 60 minutes for verification. New accounts get up to 200 USD in credits.
2. Verify student status on AWS Builder Center. Required to compete and to win.
3. Console, region us-east-1: Bedrock, Model access, request Amazon Nova Lite and Nova Pro.
4. SNS, Text messaging, sandbox: add both phone numbers, enter the OTPs. Sandbox allows up to 10 verified numbers.
5. IAM: create a user with programmatic access, download the keys. Set a billing alarm at 10 USD.
6. Request hackathon credits from the organisers once the account exists.
7. Create the public GitHub repo now so the first commit is dated inside the event window.

Local machine (backend)

- Install uv and pin the backend to Python 3.12. Python 3.14 is too new for geospatial wheels such as rasterio.
- Install the AWS CLI and the AgentCore starter toolkit. Docker is already installed.
- pip: strands-agents, strands-agents-tools, boto3, rasterio, shapely, httpx, fastapi (local dev).

Local machine (frontend)

- Node 24 is installed. create-next-app, Tailwind, shadcn/ui, maplibre-gl, Amplify CLI.

8. Known risks and the plan for each

Risk	Plan
SMS to Indian numbers normally needs DLT registration	SNS sandbox sends to verified numbers on default routes without DLT. Verify both phones today. Fallback: email via SNS and in-app alert.
Bedrock model access not approved	Request Nova models first thing. Nova access is usually instant. Fallback: any other available vision model in Bedrock.
AgentCore Runtime setup eats time	Hard 2-hour time-box. Fallback: agent runs inside Lambda. Either way Strands counts as AWS open source.
Overpass API rate limits	Pull roads and drains once for the area, commit the JSON to the repo.
Copernicus DEM 30 m tiles are large	Download only the one or two tiles covering Central Delhi, clip once, cache the clipped raster in S3.
No rain on demo day	Storm replay toggle, clearly labelled, using Open-Meteo's historical archive.
Video runs long or misses features	Script below. Record Sunday morning. Everything that should score must appear on screen.

9. Demo video script (3:00)

Time	What is on screen
0:00 - 0:20	Anita at Minto Bridge in waist-deep water. One sentence on the two gaps: no early warning, no drain information.

Time	What is on screen
0:20 - 1:00	The map with the storm replay running. Tap a red segment. The reason sentence: '60 mm coming, lowest point on the road, 2 drains nearby blocked'.
1:00 - 1:40	Upload a blocked-drain photo on a phone. Agent verifies it (Strands + Bedrock). Segment turns red. SMS arrives on a second phone, on camera.
1:40 - 2:20	Engineer screen. Counterfactual line. Mark D-14 cleaned, map improves. Hit-rate against the named Delhi hotspots.
2:20 - 2:50	Architecture diagram with every AWS service. Three-second console flashes: AgentCore, EventBridge, Step Functions, CloudWatch dashboard.
2:50 - 3:00	Data sources and freshness, stated plainly. Live URL and repo.

10. Sources

- Event page: wemakedevs.org/aws/env. Rules: wemakedevs.org/aws/env/rules. Schedule: wemakedevs.org/aws/env/schedule
- Announcement blog: wemakedevs.org/blogs/announcing-environmental-hacks
- First Commit winners: wemakedevs.org/aws/first-commit/projects. Recap: wemakedevs.org/blogs/first-commit-hackathon-recap-and-whats-next
- Build It winner writeup: builder.aws.com (Beacon Night Shift, by Prashant Thakur)
- Strands Agents: strandsagents.com. AgentCore: docs.aws.amazon.com/bedrock-agentcore
- SNS SMS sandbox: docs.aws.amazon.com/sns/latest/dg/sns-sms-sandbox.html
- Copernicus DEM on AWS: registry.opendata.aws/copernicus-dem. Open-Meteo: open-meteo.com
- Amazon Location Service with MapLibre: docs.aws.amazon.com/location